## 0 - introduction:

**Component of ML** : { · Representation · Loss function · Optimization method } → for param estimation / model selection / hyperparam tuning

feature set and model form

ex: rep: $\hat{y} = f(x;w) = w^T x$ loss: $\mathcal{L}(y,\hat{y}) = \|y-\hat{y}\|_2$ opt: $\text{argmin}_w \mathcal{L}(y,\hat{y})$, GD

**Types of Learning** :
- i) **Supervised** : x, y — Given an observation x, what is the best label y ? And self-supervised (ex: masking, diffusion models
- ii) **Unsupervised** : x — Given a set of $x_i$'s, embed or cluster them
- iii) **Reinforcement**: Given a sequence of states x and possible actions a, learn which action maximizes reward

**Types of Models** :
- i) **Generative** $p(x)$
- ii) **Discriminative** $p(y|x)$ ↳ "conditional models" ?

- i) **parametric** $\hat{y} = w \cdot x$ ; $\hat{y} = f(x;\theta)$ (w and θ are params)
- ii) **non-parametric** k-nn, decision trees
- iii) **semi-parametric** deep learning

X: features, predictors, design matrix, input
y: response, label, output

(btw: X is nxp ; $X^TX$ is hence pxp) y is p×1 ?

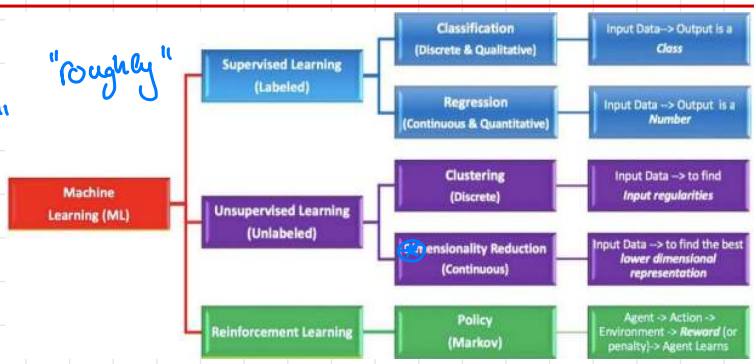
---

## 1b - Non parametric

"cool word for dimensionality reduction is **embedding**"

"PCA is one form of dimensionality reduction"

discrete ≠ qualitative




- $\hat{y} = f(x;\theta)$ (estimate, model, param, input feature)

**Empirical risk**: $\hat{R}(\theta) = \frac{1}{n}\sum_{i=1}^{n} l(\hat{y}_i, y_i)$ → "average loss over the training points"
↳ "the risk on the training data"

"We don't care so much about minimizing the past (empirical) risk, we care more about the future risk !"

**True risk**: $R(f) = \mathbb{E}_{(x,y)\sim P}[\mathcal{L}(f(x),y)]$ (iid assumption)! → the expected loss over the true (unknown) data distribution

**Bayes Optimal**: $f^*(x) = \text{argmin}_a \mathbb{E}[\mathcal{L}(a,y)|x]$ → "best possible predictor given the data distribution"

↑ "here, $a$ is a particular $\hat{y}$"

↑ you can get two different $y$'s for the same $x$! Hence, no matter how good your function is, you'll never get it perfectly ↑ => ↑ error? is never 0 ↑

- **K-NN**: (K nearest neighbor) is a classification and regression algo:
  1) find the K nearest neighbor (according to a distance metric $d(\cdot,\cdot)$)
  2) $\hat{y}(x)$ is the majority label or the average of the $y$'s of those points

↳ K-NN is **nonparametric** but has two hyperparameters: K and $d(\cdot,\cdot)$ >?

- **$L_p$ norm**: $\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{\frac{1}{p}}$ ; where $x = (x_1, \dots, x_n)$ is a vector $(p \geq 1)$

↳ properties: i) $\|a\underline{v}\|_p = |a|\,\|\underline{v}\|_p$ (homogeneity) iii) $\|\underline{v}\| = 0 \iff \underline{v}$ is the zero vector iv) $\|x\|_p \geq 0$ ?

ii) $\|\underline{u}+\underline{v}\|_p \leq \|\underline{u}\|_p + \|\underline{v}\|_p$ (triangle inequality or subadditivity) ?✓

- for $0 < p < 1$ ; $L_p$ fails the triangle ineq., hence it's not a true norm!

- **$L_0$ pseudo-norm** $\|x\|_0 := \#\{i : x_i \neq 0\}$ i.e. the number of nonzero components of $x$

↳ It's not a true norm since it fails homogeneity

- **$L_\infty$**: $\|x\|_\infty = \max_i |x_i|$ i.e. the largest absolute component of $x$ $\left(= \lim_{p\to\infty} \|x\|_p\right)$

- "Every norm generates a distance, they kinda have the same properties"

$d_p(x,y) = \|x-y\|_p$ ; distance properties:

i) $d(x,y) \geq 0$ (non-negativity or separation axiom)

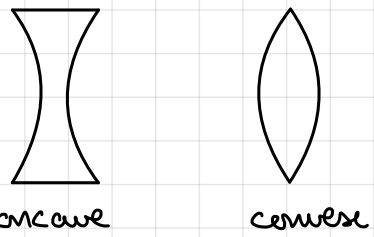
ii) $d(x,y) = 0 \iff x = y$ (coincidence axiom)

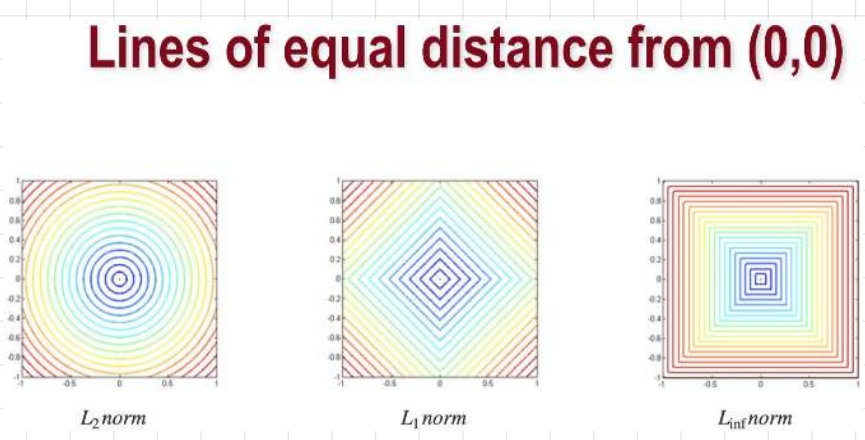
iii) $d(x,y) = d(y,x)$ (symmetry)

iv) $d(x,z) \leq d(x,y) + d(y,z)$ (triangle ineq.)

- **Convexity**: A figure is convex if any line segment connecting two points on the surface of the figure lies entirely inside the figure; otherwise it's concave ↑

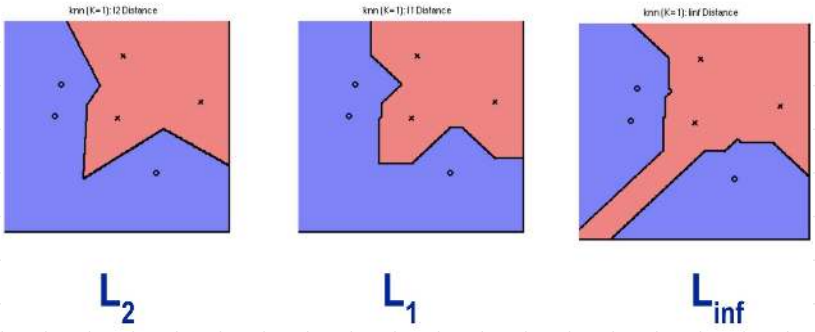
ex:



rule of thumb: "Convex problems are easy to solve with GD, anything non-convex is hard, you gotta be lucky ..." "convexity guarantees global optima, unless you're stupid"

## Different norms give different decision boundaries (For K-nn)




subtlety: "kNN is nonparametric, it has no optimization, no training phase, no parameters (it has hyperparameters), hence no explicit loss function. Implicitly, it locally minimizes: { squared loss for regression; 0-1 loss for classification"

subtlety: difference between parameters and hyperparameters (by chat GPT)

- parameters: the internal variables the model learns from data during training. they are found by minimizing the loss function on the training data
- hyperparameters: External settings chosen before training that control how the model learns or behaves. they are not learned from the data directly, we choose them via validation or cross-validation (see later). they define the learning process.

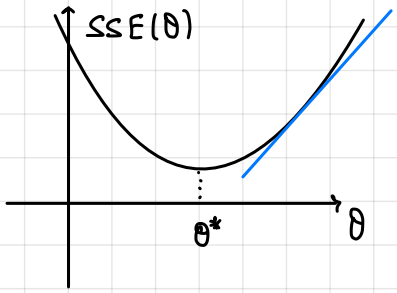
- In high dimensions, most points are equally close to each other, because variance of distances shrink relative to their mean. this reduces the usefulness of distance metrics, and is known as the curse of dimensionality

## 2a - Gradient descent :

$$\theta_{i+1} = \theta_i - \eta \nabla_\theta \mathcal{L}(\theta)$$

"$\nabla_\theta$ tells us which direction to go, $\eta$ tells us how fast to go"

- $\eta$: learning rate, "eta", it's a hyperparameter
- $\mathcal{L}$: loss function




this is in 2 dimensions, we can generalize to d-dimension

An example of a loss function is the

**Sum of Squared Errors** :

$$SSE(\theta) := \sum_{i=1}^{n} (f_\theta(x_i) - y_i)^2$$

where $r_i(\theta) = f_\theta(x_i) - y_i$ are the residuals

hence $SSE(\theta) = \sum_{i=1}^{n} r_i(\theta)^2$

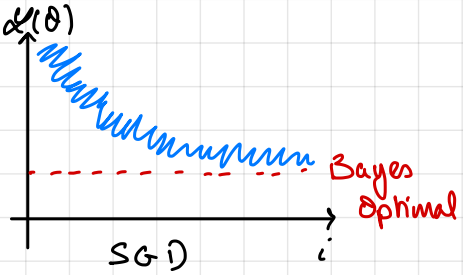
"We'll assume our loss functions are in general continuous and differentiable"

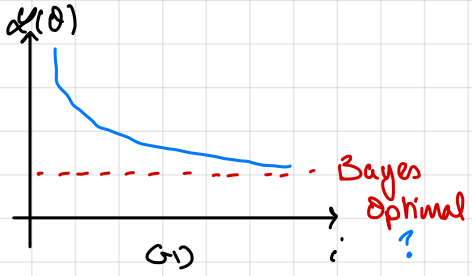
"If given the choice between analytic and numerical differentiation, choose analytic!"

btw, **Mean Square Error** $(MSE) := \frac{1}{n} SSE$, and **Root Mean Square Error** $(RMSE) := \sqrt{MSE}$

- **Stochastic Gradient descent** (SGD):

"If we have a large dataset, update the model after observing each single observation"




- **Mini-Batch**: "Update the model every $k$ observations (ex: batch size $k = 50$)" (*)

↳ this is more efficient than SGD or full GD, and what DL practitioners use.

- **Momentum**:

$$\theta_{t+1} = \theta_t - \eta v_{t+1}, \text{ where } v_{t+1} = \rho v_t + \nabla_\theta \mathcal{L}(\theta_t)$$

"Continue to move in the direction you were moving"

↳ $v$: "velocity", initialized at $v_0 = 0$ ; $\rho$: "momentum coefficient"

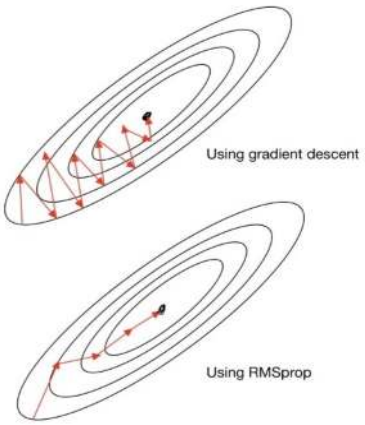
- **Root Mean Square Propagation (RMS Prop)**:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_{t+1} + \varepsilon}} \nabla_\theta \mathcal{L}(\theta_t)$$

where $s_{t+1} = \rho s_t + (1-\rho)(\nabla_\theta \mathcal{L}(\theta_t))^2$

$s$: moving average of squared gradient → initialize at 0 before the first iteration

↳ smaller steps for large gradients, larger steps for small gradients




$\rho$: decay rate (ex: 0.9) ; $\varepsilon$: small constant to avoid division by 0 (ex: $10^{-8}$)

(*) you want enough of them to fill your GPU, else you're wasting GPU cycles

• **Adam**: "Momentum + RMS prop" "don't worry about the details"

↳ "good place to start!"

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}} + \varepsilon} \hat{m}_{t+1}$$

where

$$m_{t+1} = \beta_1 m_t + (1-\beta_1) \nabla_\theta \mathcal{L}(\theta_t)$$

$$v_{t+1} = \beta_2 v_t + (1-\beta_2) \left(\nabla_\theta \mathcal{L}(\theta_t)\right)^2$$

;

$$\hat{m}_{t+1} = \frac{m_{t+1}}{1-\beta_1^{t+1}}$$

$$\hat{v}_{t+1} = \frac{v_{t+1}}{1-\beta_2^{t+1}}$$

} unbiased $m$ & $v$

↳ $\beta_1, \beta_2$ are decay rates

• **Adagrad**: Define a per-feature learning rate for feature $j$ as: $\eta_{t,j} = \frac{\eta}{\sqrt{G_{t,j}}}$ where $G_{t,j} = \sum_{k=1}^{t} g_{k,j}^2$

where $g_k := \frac{\partial}{\partial \theta_j} \text{cost}_\theta(x_R, y_R)$

$\overset{?}{\Rightarrow}$ $\theta_j \leftarrow \theta_j - \frac{\eta}{\sqrt{G_{t,j}} + \varepsilon} g_{t,j}$

↳ $G_{t,j}$ is the sum of squares of gradients of feature $j$ over time $t$

↳ Frequently occuring features in the gradients get small learning rates, rare features get higher ones.

↳ key idea: "learn slowly" from frequent features, but "pay attention" to rare but informative features

• **Coordinate descent**: use when we're in $L_1$ world ? ...

"we'll mostly not see it"...

→ didn't really mention adamw, adagrad, coord descent ...

and ill-conditionning

# 3a - Regression:

- **Maximum Likelihood Estimation (MLE):** $\underset{\theta}{\text{argmax}}\ p(\mathcal{D}|\theta)$ (data, param)
- **Maximum a-posteriori (MAP):** $\underset{\theta}{\text{argmax}}\ \underbrace{p(\mathcal{D}|\theta)}_{\text{likelihood}}\underbrace{p(\theta)}_{\text{prior}} \propto p(\theta|\mathcal{D})$

recall Bayes rule:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)\,p(\theta)}{p(\mathcal{D})}$$

likelihood, prior, posterior, marginal

- An estimator is "a rule for computing estimates of a parameter $\theta$. An estimator is **consistent** (or asymptotically consistent) if as the number of data points used increased indefinitely, the resulting sequence of estimates converges in probability to the true parameter $\theta$.
- **Both MLE and MAP are consistent** right?
- ↑ MLE is a special case of MAP ↑ (with uniform prior?)

Prof. Lyle in practice never uses MLE, it's bad apparently, he always uses MAP

MLE is provably the best to fit the training data, but he only cares about the test data

- In practice, we don't care about asymptotic behavior of estimators, because we never have infinite data to work with

ex: Prof. Lyle worked at Google, with the whole internet as his dataset, and he didn't really care about asymptotic properties.

In theory, they are nice to have.

- Also, often $X^TX$ is singular $\Rightarrow$ no MLE ...
- Linear regression assumes that the expected output given an input $x$, $\mathbb{E}[y|x]$, is linear in $x$.

**I) One-parameter linear regression:** $\boxed{y_i = wx_i + \varepsilon_i}$, where $\varepsilon_i \overset{iid}{\sim} \mathcal{N}(0, \sigma^2)$ with $\sigma^2$ unknown (noise)

$\Rightarrow\ y(x) \sim \mathcal{N}(wx, \sigma^2)$

We want to estimate $w$ from the data $\mathcal{D}$

We can find a posterior for $w$ given $\mathcal{D}$ using Bayes rule (this is **Bayesian Linear Regression**)

Or we can use MLE ↓

$$\hat{w}_{MLE} = \underset{w}{\operatorname{argmax}}\, p(\mathcal{D} \mid w, x_1, \dots, x_n) \overset{iid}{=} \underset{w}{\operatorname{argmax}} \prod_{i=1}^{n} p(y_i \mid w, x_i)$$

$$= \underset{w}{\operatorname{argmax}} \prod_{i=1}^{n} \exp\left(-\frac{(y_i - wx_i)^2}{2\sigma^2}\right)$$

$$= \underset{w}{\operatorname{argmax}} \sum_{i=1}^{n} -\frac{1}{2}\left(\frac{y_i - wx_i}{\sigma}\right)^2$$

$$= \underset{w}{\operatorname{argmax}} \overset{?}{-} \sum_{i=1}^{n} (y_i - wx_i)^2$$

⇒ MLE with Gaussian noise is the same as minimizing the $L_2$ error $\quad \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - wx_i)^2$

$\hat{w}_{MLE}$ is the one that minimizes the sum of squares residuals $r_i = y_i - wx_i$ ; $\hat{w}_{MLE} = \dfrac{\sum_{i=1}^{n} x_i y_i}{\sum_{i=1}^{n} x_i^2}$

- What about MAP ? Same assumption $y_i \sim \mathcal{N}(wx_i, \sigma^2)$ and add a prior $w \sim \mathcal{N}(0, \gamma^2)$

$$\hat{w}_{MAP} = \underset{w}{\operatorname{argmax}} \prod_{i=1}^{n} p(y_i \mid w, x_i)\, p(w) = \dots = \underset{w}{\operatorname{argmin}} \sum_{i=1}^{n} (y_i - wx_i)^2 + \lambda w^2 \quad \text{where } \lambda = \frac{\sigma^2}{\gamma^2}$$

↓

MAP with Gaussian prior on $w$ is the same as minimizing the $L_2$ error plus an $L_2$ penalty on $w$.

this is called <u>ridge regression</u> (or <u>shrinkage</u>, or <u>Tikhonov regularization</u>)

## II) Multivariate Linear Regression:

$\hat{w}_{MLE} = (X^T X)^{-1} X^T Y$ ; $\hat{w}_{MAP} = (X^T X + \lambda I)^{-1} X^T Y$ btw $X$

btw

- Linear data often doesn't go through the origin ⇒ create a fake input $x_0$ that is always 1
- $L_1$ regression : $\hat{w} = \underset{w}{\operatorname{argmin}} \|y - Xw\|_1$ (this is still convex!)
- A method is <u>scale invariant</u> if multiplying any feature (any column of $X$) by a nonzero constant does not change the predictions made (after retraining)

does your model change if you rescale the variables, (ex: change the units from dollars to cents or miles to kilometers)

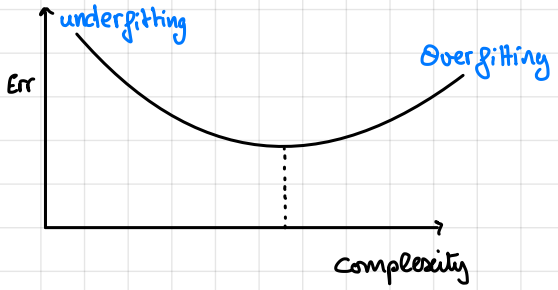
i.e. linear regression ?

| | |
|---|---|
| scale invariant | OLS ($L_2$ regression) ; decision trees |
| not scale invariant | k-NN, ridge regression |

↳ because it preferentially shrinks "large" coeffs

"unless features are standardized"

# 3b - Regularization:




↱ the more complex the model, the better it fits the data, but I don't care about that, I care about future data ↰

↱ the simpler the model, the less well it's gonna fit the data, but also the less it is gonna vary for different data sets. (i.e. the more stable it's gonna be) ↰

↱ the more complex ——— more ——— more ——— less ———

↳ simpler model: higher bias, lower variance  complex model: lower bias higher variance

↱ a trade-off between bias and variance

- Test error = Train error + Model complexity (other formula below)
- Training set ("in sample"): the data used to fit the model (learn the params)
- Testing set ("out of sample"): data not seen during training, used to estimate how well the model generalizes to new data
- K-Fold Cross Validation: Divide the dataset k-fold, use k-1 folds for training and 1 for testing, repeat for each fold (i.e. k times), average the error: $\frac{1}{k}\sum_{i=1}^{k} \text{Error}_i$

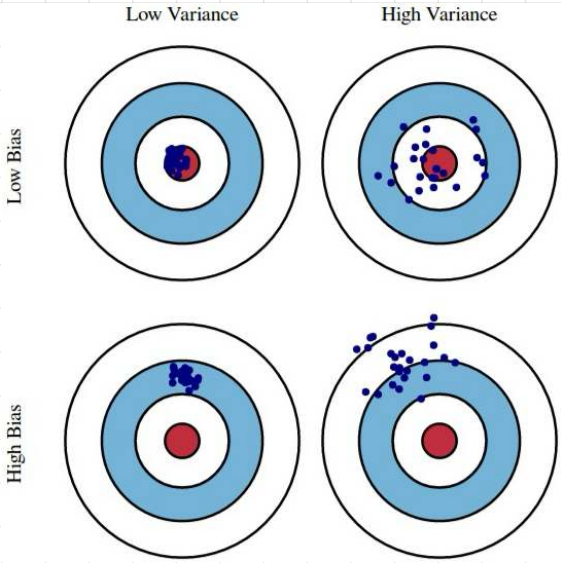
↳ is used to pick hyperparameters and estimating ~~true~~ test! error ?

↳ ex: k = 10 or 5 (common) or k=n (where n is the number of data points) is called leave-one-out cross validation (LOOCV)

↳ consider cases with or without replacement ? not important ?

- $\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta} - \theta] = \mathbb{E}[\hat{\theta}] - \mathbb{E}[\theta]$
- $\text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]$
- Test Error = Variance + Bias$^2$ + Noise

↱ In the limit, ridge reg shrinks the params to 0 hence 0 variance ↰ ↱ OLS linear regression is unbiased in $\hat{\theta}$ or $\hat{y}$, ridge makes it biased ↰



ex: OLS - (Ordinary least Squares)

since $y \sim \mathcal{N}(X\beta, \sigma^2)$

$\hat{\beta} = (X^TX)^{-1}X^Ty$ ; $\mathbb{E}[\hat{\beta}] = (X^TX)^{-1}X^T\mathbb{E}[y] = (X^TX)^{-1}X^T(X\beta) = \beta \Rightarrow Bias(\hat{\beta}) = 0$

↳ if $(X^TX)^{-1}$ exists

similarly; $\mathbb{E}[\hat{y}] = \mathbb{E}[X\hat{\beta}] = \dots = X\beta \Rightarrow Bias(\hat{y}) = 0$

In the real world, variables are usually correlated, hence we don't usually use unbiased estimators in practice, because variance goes up fast

**Penalties:** A good way to control bias / variance

ex: ridge : $\|y - Xw\|_2^2 + \lambda\|w\|_2^2$ ; minimizing $\lambda\|w\|_2^2$ will reduce the variance, but will also increase bias

minimizing $\|y - Xw\|_2^2$ will decrease both (OLS ?)

↳ we call the second term a regularization penalty ($\lambda\|w\|_2^2$ in the previous case)

we used $L_2$ penalty, but could have used $L_1$ or $L_0$ (btw define $0^0 = 0$ here)

All of these norms as penalties encourage the $w_j$'s to be smaller, they shrink $w$.

If we want $\underset{w}{\operatorname{argmin}} \underbrace{\|y - Xw\|_2^2}_{\text{Error}} + \underbrace{\lambda\|w\|_p^p}_{\text{penalty}}$ ⇒

- if $p = 2$, (ridge regression), convex, makes the w's on average a little smaller
- if $p = 1$, (LASSO), convex, drives some w's to 0 (feature selection)
- if $p = 0$, (stepwise regression), nonconvex ⇒ requires search !

$p = 2$ : Gaussian prior

$p = 1$ : Laplace prior ?

$p = 0$ : spike and slab prior

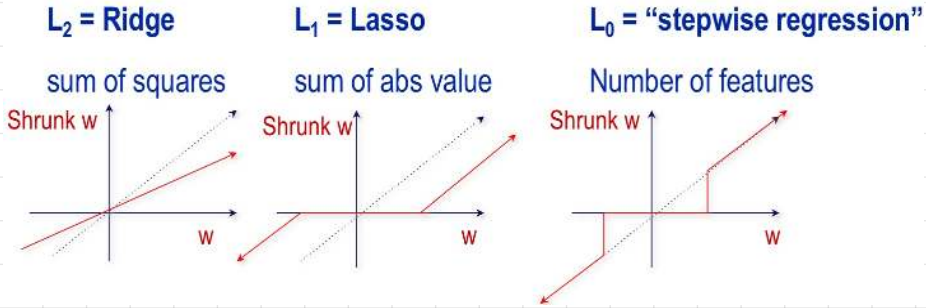
How to solve

$p = 2$ : $(X^TX + \lambda I)^{-1}X^Ty$

$p = 1$ : Gradient descent

$p = 0$ search (stepwise or streamwise)

- If *the x's have been standardized* (mean zero, variance 1) then we can visualize the shinkage:




- $L_2$ penalty most heavily shrinks large weights
- $L_0$ penalty most strongly encourages weights to be 0
- $L_0$ penalty is scale invariant, $L_1$ & $L_2$ are not

## Stepwise regression

- **Initialize:**
  - model = {},
  - $Err_{old} = \Sigma_i (y_i - 0)^2 + 0$
- **Repeat (up to *p* times)**
  - **Try** adding each feature $x_k$ to the model
    - Pick the feature that gives the lowest error
    - $Err = min_k \Sigma_i (y_i - \Sigma_{j \text{ in model}k} w_j x_{ij})^2 + \lambda |model_k|$
  - ***If*** $Err < Err_{old}$
    - Add the feature to the model
    - $Err_{old} = Err$
  - ***Else*** Halt

↳ less greedy, but still greedy

it's what stats people use usually

## Streamwise regression

- **Initialize:**
  - model = {},
  - $Err_0 = \Sigma_i (y_i - 0)^2 + 0$
- **For each feature $x_j$, j=1:p:**
  - **Try** adding the feature $x_j$ to the model
  - ***If***
    - $Err = \Sigma_i (y_i - \Sigma_{j \text{ in model}} w_j x_{ij})^2 + \lambda ||model||_0 < Err_{j-1}$
    - Accept new model and set $Err_j = Err$
  - ***Else***
    - Keep old model and set $Err_j = Err_{j-1}$

$||model||_0$ = # of features in the model

↳ it's a greedy search, won't find the best solution in general

- To choose $\lambda$, search over $\lambda$'s to minimize the non-penalized error on a test (or CV error)
- Btw: $\underset{w}{\text{argmin}} \; ||y - Xw||_2^2 + \lambda_2 ||w||_2^2 + \lambda_1 ||w||_1$ ($L_2$, $L_1$) is called Elastic Net

is one of the most used ML models in practice. Is convex!

- didn't mention: sparsity, ↑ important

↳ penalized error approximates test error ↑

# 4a - Generalized Linear Models (GLM) and Radial Basis Functions (RBF)

I)

- **Logistic Regression**: When y is binary $\in \{-1, +1\}$ — used for binary classification, despite the name haha! (in practice, very simple and useful!)

**Logistic function**: $h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$ $\stackrel{?}{=}$ **sigmoid**: $\sigma(z) = \frac{1}{1 + e^{-z}}$

$$p(Y=1 \mid x, w) = p(Y=-1 \mid x, w) = \frac{1}{1 + e^{-y w^T x}} = 1 - p(Y=1 \mid x, w) \Rightarrow \boxed{w^T x = 0}$$

defines a hyperplane! — decision boundary

score vector ?

$\Rightarrow$ to classify: $h(x) = \text{sign}(w^T x) = \begin{cases} 1 & \text{if } w^T x > 0 \\ -1 & \text{if } w^T x < 0 \end{cases}$

We can take the **log odds**: $\boxed{\log \frac{p(Y=1 \mid x, w)}{p(Y=-1 \mid x, w)} = w^T x}$ ← linear in x's and w (hard to optimize, but nice to think about)

MLE: $\log(p(D \mid x, w)) = \log\left(\prod_i \frac{1}{1 + e^{-y_i w^T x_i}}\right) = -\sum_i \log(1 + e^{-y_i w^T x_i})$ is convex! so we're good

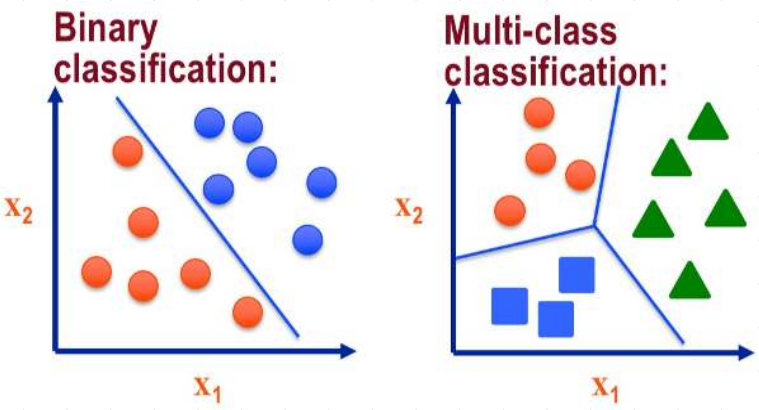
MAP: ... (I'll skip for now, see slides)

↳ TO compute MLE & MAP, use gradient descent!

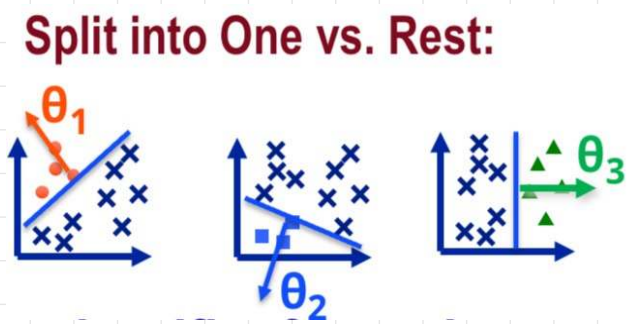
- **Multi-Class Classification**: for K classes,

$$\boxed{p(y=k \mid x, \theta_1, \ldots, \theta_K) = \frac{e^{\theta_k^T x}}{\sum_{k=1}^{K} e^{\theta_k^T x}}}$$ → normalizer to get a valid proba distr.

is the **softmax function**, which maps a vector to a probability distribution




- Train a logistic regression classifier for each class k to predict $p(y=k \mid x)$ using the softmax
- Gradient descent simultaneously updates all params for all models
- predict class label as the most probable label



II]

**Linear Basis Function Models**: Generally, $\hat{y}(x) = \sum_{j=1}^{M} \theta_j \phi_j(x)$, where $\phi_j(x)$ are basis functions.

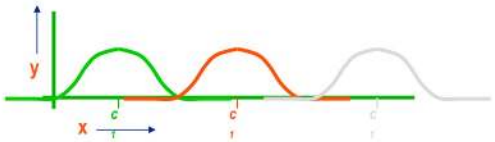
Typically, $\phi_0 = 1$ so that $\theta_0$ acts as a bias.

In the simplest case, we use linear basis functions: $\phi_j(x) = x_j$ but we could also use

polynomial basis functions: $\phi_j(x) = x^j$ (global, mostly crappy) or Gaussian basis functions: $\phi_j(x) = e^{-\frac{(x-\mu_j)^2}{2s^2}}$ (local, good!)

⊕ section on RBFs ---

## 1-D RBFs




$\hat{y} = w_1\phi_1(x) + w_2\phi_2(x) + w_3\phi_3(x)$

where

$\phi_i(x) = k(\|x - \mu_j\| / C)$

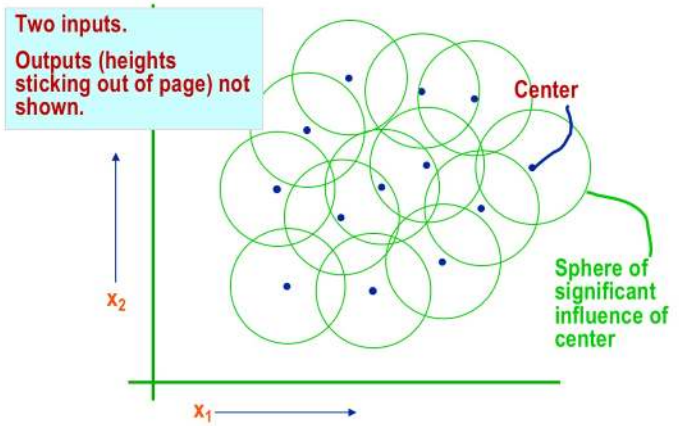
For RBF:

$k(\|x - \mu_j\| / C) = \exp\{-\|x - \mu_j\|_2^2 / C\}$

C = "Kernel Width"

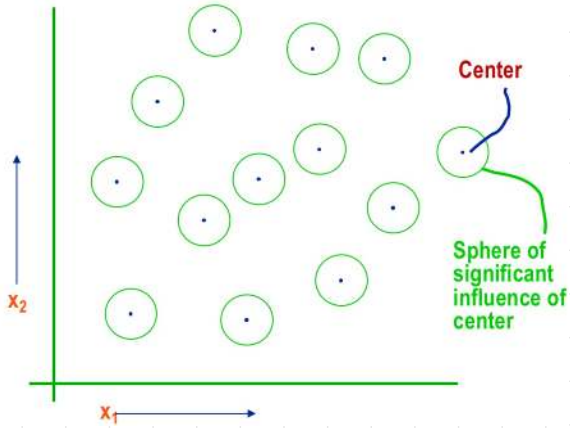
k = kernel function

## Radial Basis Functions in 2-d

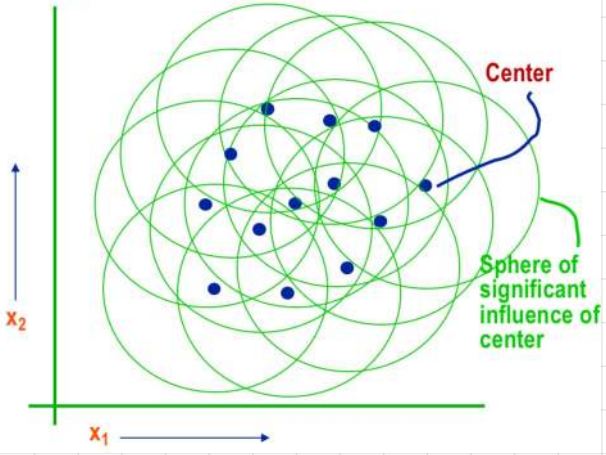



## Too small!

Even before seeing the data, you should understand that this is a disaster!




## Too big!!!



# So what do we do?

Search to find the optimal size "width" for the Gaussians (on a test set, of course!)

# RBFs can do …

- **Use d < p basis vectors**
  - Dimensionality reduction
  - Good for high dimensional feature spaces
- **Use d > p basis vectors**
  - Increases the dimensionality
  - Can make a formerly nonlinear problem linear
- **Use d = n basis vectors**
  - We can use this to switch to a *dual* representation (later!!)

# How to find the kernel centers?

- **Pick random points**
  - Generally a bad idea
- **Standard RBF: do k-means clustering and use the centers of the clusters**
  - Works great!
- **Use all *n* of the training data points as kernel centers**
  - Requires regularization
- **Estimate them: nonlinear regression**
  - A good initialization helps

# What you should know (2)

- **Link functions give a nonlinear regression**
  - e.g. logistic regression
- **Feature transformations allow one to fit a nonlinear function using linear regression**
- **RBF**
  - Cluster points
  - Put a Gaussian basic function at each cluster center
    - Pick the Gaussian width
  - Fit a linear regression

# 4b - Kernels and Support Vector Machines

- A **kernel** (function) measures the similarity ($= k(x, y)$) between two points $x$ and $y$

↳ it has a corresponding **kernel matrix** $K$ which is **symmetric and positive semi-definite** (SPSD) ($\Leftrightarrow$ PSD)

where $K_{ij} = k(x_i, x_j)$ from all pairs of rows of a matrix $X$

btw: a PSD matrix has only non-negative eigenvalues (i.e. $\geq 0$)

**uses of kernels:** RBF, kernel regression, SVMs, ...

**examples of kernels:** i) linear kernel: $k(x, y) = x^T y$ (simple dot product)

ii) Gaussian kernel: $k(x, y) = e^{-\frac{\|x-y\|_2^2}{c}}$ iii) quadratic kernel: $k(x, y) = (x^T y)^2$ or $(x^T y + 1)^2$

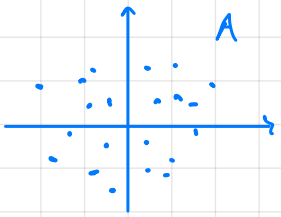
↳ has a hyperparam "$c$" called the **width of the kernel** !

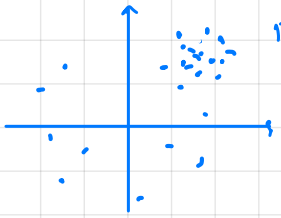
→ skip slide on RBF review

- **kernel regression:** $\hat{y}(x) = \frac{\sum_{i=1}^{n} K(x, x_i)\, y_i}{\sum_{i=1}^{n} K(x, x_i)}$ ("target $x$", $x_i$ in the data) ; **kernel classification:** $\hat{y}(x) = \text{sign}\left(\sum_{i=1}^{n} K(x, x_i)\, y_i\right)$

↳ this regression has no gradient descent, no parameters to be estimated, all we need is the kernel function (no optimization)

**kernel regression v/s k-nn:**




If we have case B, we rather use k-nn

If we have case A, we rather use kernel regression!

"kernels assumes similarity is the same everywhere"

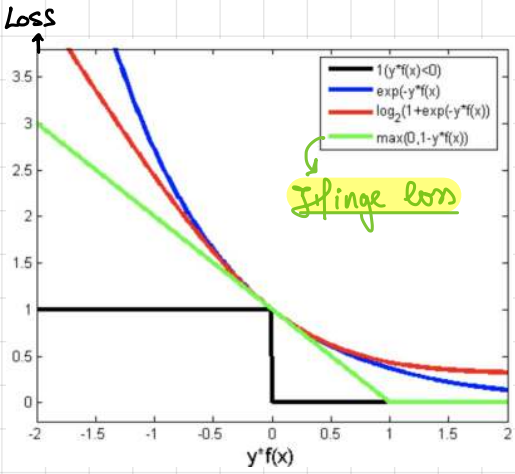
ex: Let $X_{3\times 2} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$ and assume $K(\cdot,\cdot)$ is the linear kernel ; (dot product)

then the matrix $K_{3\times 3} = \begin{bmatrix} 5 & 11 & 17 \\ 11 & 25 & 39 \\ 17 & 39 & 61 \end{bmatrix}$ since $\begin{cases} x_1 = [1, 2] \\ x_2 = [3, 4] \\ x_3 = [5, 6] \end{cases}$ and $K = \begin{bmatrix} x_1^T x_1 & x_2^T x_1 & x_3^T x_1 \\ x_1^T x_2 & x_2^T x_2 & x_3^T x_2 \\ x_1^T x_3 & x_2^T x_3 & x_3^T x_3 \end{bmatrix}$

↳ each row is an observation $x_i$

# • Support Vector Machines: (used for classification)

This page incomplete




i.e. $\boxed{\text{hinge loss} = \max(0,\ 1 - y^* f(x))}$

Clearly, the hinge loss is the least sensitive to the mislabeled or noisy samples

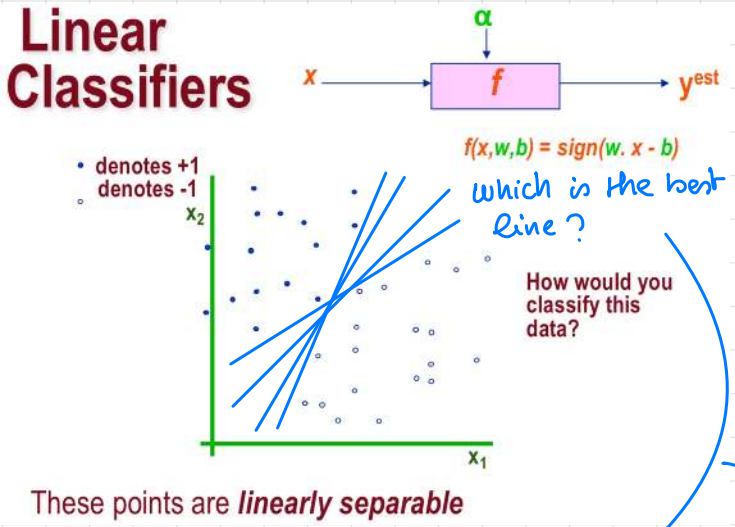
ridge penalty — regularization constant — hinge loss

$$\text{SVM}: h(x) = \text{sign}(w^T x + b) \quad ; \quad \min_{w,b,\xi \geq 0} \frac{1}{2} w^T w + C \sum_i \xi_i \quad \text{where } \xi_i = \max(0,\ 1 - y_i (w^T x_i + b))$$

decision boundary

0 if score is correct by 1 or more

"Hinge loss doesn't prevent you from overfitting, you need to shrink weights to avoid overfitting"

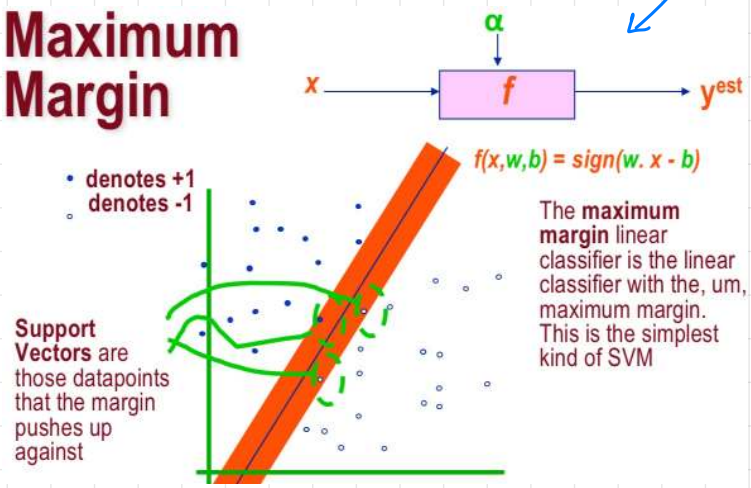



linearly separable means there exists a hyperplane, such that you get a loss of 0 on the training set

(not realistic in practice, except if we overfit)

→ this is what the hinge loss does?

define the margin of a linear classifier as the width that the boundary could be increased by before hitting a datapoint




• btw SVMs is not scale invariant!

Separable SVM primal: $\min_{w,b} \frac{1}{2} w^T w$ s.t. $y_i(w^T x_i + b) \geq 1$ for $i = 1, \dots, n$

this maximizes margin — "at least 1 away"

- "Points that are on the correct side of the margin don't contribute to the hinge loss function. So the loss is entirely in terms of the points inside the margin or on the wrong side of it."

General SVM: $\min_{w,b,\xi \geq 0} \frac{1}{2} \|w\|_p^p + C \|\xi\|_q^q$ s.t. $y_i(w^T x_i + b) \geq 1 - \xi_i$ ; $i = 1, \dots, n$

SVM can also be solved in the dual: $\max_{\alpha \geq 0} \sum_i \alpha_i - \frac{1}{2} \sum_{ij} \alpha_i \alpha_j y_i y_j x_i^T x_j$

s.t. $\sum_i \alpha_i y_i = 0$ ; $\alpha_i \leq C$ ; $i = 1, \dots, n$

note:
- $x_i^T x_j$ is a kernel matrix, it can be replaced with any kernel
- $C$ controls regularization
- Most $\alpha_i$ are 0 (points where hinge loss is 0)

- Kernel SVMs: high $C$ ⇒ big penalty for misclassified data

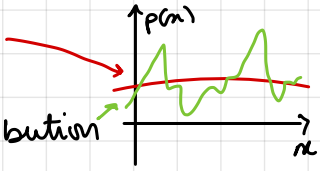
↳ (in Gaussian) high $\gamma$ ⇒ only closed neighbors have influence ?

↳ nice for data that are not linearly separable

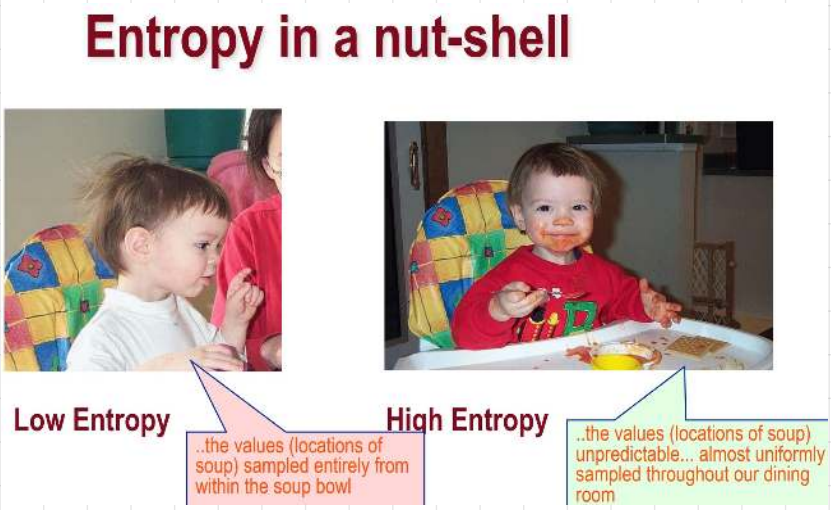
# 5a - Information theory and Decision Trees:

$\log_2$ ↓ ← not e ↓ watch the minus sign!

the **entropy** of a random variable $X$ is $H(X) = -\sum_{i=1}^{m} p_i \log_2(p_i)$ where $m$ is the number of events $E_i$ that $X$ can take and $p_i$ their associated probability: $P(E_i) = p_i$




- high entropy ⇒ $X$ follows a ~ uniform-ish, boring distribution
- low entropy ⇒ $X$ follows a varied (peaks and valleys) distribution
- entropy $\in [0, +\infty)$ but $\in [0, 1]$ if $X$ is a boolean. (special case)
  ↳ note entropy is never negative
- entropy = 0 ⇒ an event is certain
- two events are equally likely ⇒ $H(X) = 1$




- "the better the model, the lower the entropy of $Y|X$"

↳ **Conditional entropy**: $H(Y|X) = \sum_{i=1}^{m} P(X=E_i)\, H(Y|X=E_i)$ = the average conditional entropy of $Y$

- **Information gain**: $IG(Y|X) = H(Y) - H(Y|X)$

↳ "usually the more a random variable has different labels, the more info it gives you"

In practice, data is expensive, hence we use IG to know which data is the most informative. (ex: the best question to ask next in a decision tree is the one with highest IG). IG is also useful for model selection.

## Decision Trees:

→ slide 23 →

the standard recursive partition to maximize info gain ---

KL divergence $D_{KL}(P||Q) = \sum_j p(x_j) \log \frac{p(x_j)}{q(x_j)}$

↳ measures how ≠ two distros are

$= -\sum_i p(x_j) \log(q(x_j)) + \sum_j p(x_j) \log(p(x_j))$

$D_{KL} = H(P,Q) - H(P)$

neither a distance nor a similarity.

$D_{KL}$ = Cross entr − entr

btw: Cross entropy: $H(P,Q) = -\sum p(x) \log(q(x))$

$D_{KL} \geq 0$ and not symmetric

$D_{KL}(P||Q) = 0 \iff P = Q$

## 5b - Ensembles :

# 6a - Online learning : streaming (in observations) v/s streamwise (in features)

↳ continuously updating our model with either new observations or new features

"I have more features than observations ( less google pages than page elements )"

- Batch learning can be expensive for large datasets : (ex: to compute $(X^TX)^{-1}$, we need $\underbrace{p^2n}_{X^TX} + \underbrace{p^3}_{\text{the inverse}} \simeq O(p^3)$ computations ... )
  (the naïve way, we'll see SVD later)

"In practice you never want to invert a matrix in an ML course, it's too expensive"
↳ even the matrix multiplication sometimes ...

Online methods are well-suited for map-reduce environments.
↳ they're often clever versions of SGD

We're gonna cover two online learning methods : Least Mean Squares and Perceptron
"SGD for linear regression" "streaming hinge loss"

## Least Mean Squares (LMS) : "Online linear regression"

Minimize $Err = \sum_{i=1}^{n} (y_i - w^T x_i)^2$ using SGD (look at each observation $(x_i, y_i)$, sequentially decrease its error $Err_i = (y_i - w^T x_i)^2$

algo: $w_{i+1} = w_i - \frac{\eta}{2} \frac{dErr_i}{dw_i}$

where $\frac{dErr_i}{dw_i} = -2(y_i - w_i^T x_i) x_i = -2 r_i x_i$ (where $r_i = y_i - w_i^T x_i$ are the residuals ?)

$\Rightarrow$ $\boxed{w_{i+1} = w_i + \eta r_i x_i}$ here, $i$ indexes both the iteration and the observation, since there's only one update per observation

how to pick $\eta$ ? not CV, CV is usually for hyperparameters that control complexity
↳ "small enough for stability, large enough for fast convergence"

LMS converges for $0 < \eta < \frac{1}{\lambda_{max}}$ where $\lambda_{max}$ is the largest eigval of $X^TX$
"in general"

↳ the convergence rate is proportional to $\frac{\lambda_{min}}{\lambda_{max}}$

- btw : In practice, $X^TX$ is often not invertible, since some features might be colinear. Hence, we often don't use linear regression. Rather, we use ridge ! (add a little ridge) then, $X^TX$ is always invertible !

• Perceptron Learning Algorithm (PLA)? "Online SVM" (for classification)

# Perceptron Learning Algorithm

**Input:** A list $T$ of training examples $\langle \vec{x}_0, y_0 \rangle \dots \langle \vec{x}_n, y_n \rangle$ where $\forall i : y_i \in \{+1, -1\}$
**Output:** A classifying hyperplane $\vec{w}$
Randomly initialize $\vec{w}$;
**while** *model $\vec{w}$ makes errors on the training data* **do**
  **for** $\langle \vec{x}_i, y_i \rangle$ *in* $T$ **do**
    Let $\hat{y} = sign(\vec{w} \cdot \vec{x}_i)$; ← the score
    **if** $\hat{y} \neq y_i$ **then**
      $\vec{w} = \vec{w} + y_i \vec{x}_i$; ← If you were wrong, make w look more like x
    **end**
  **end**
**end**

What do we do if error is zero?

Of course, this only converges for linearly separable data

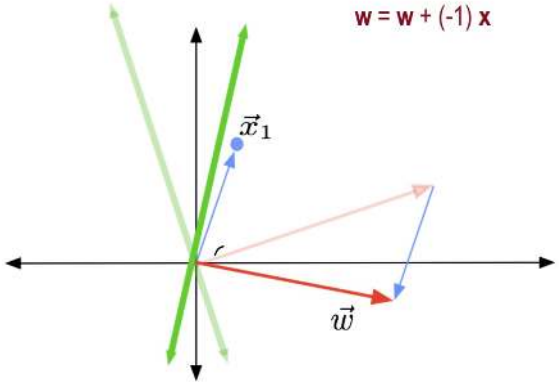
For each observation $(x_i, y_i)$,

$w_{i+1} = w_i + \eta\, r_i x_i$

where $r_i = y_i - \text{sign}(w_i^T x_i)$ and $\eta = \frac{1}{\alpha}$

i.e.: { if we got it right ($r_i = 0$) : no change !
{ if we got it wrong ($r_i = 2$) : $w_{i+1} = w_i + y_i x_i$

## Perceptron update example




here, the prediction at $x_1$ was wrong,

and this shows the update in that case.

properties of the simple perceptron:

i) If the data are linearly separable, the the algo will converge to a solution (Perceptron Convergence theorem)

ii) the number of mistakes of it makes is bounded by $\eta < \frac{R^2}{\gamma^2}$ where $R = \max_i \|x_i\|_2$ (size of biggest x)

iii) $\gamma < y_i w^{*T} x_i$ for all $i$, and $\gamma > 0$ if the data are linearly separable

$\gamma$ the margin; $w^*$ "w that perfectly classifies all points"; linearly separable often not the case in practice

→ If it's not linearly separable, then the perceptron is unstable and bounces around

voted perceptron: Like a regular perceptron, but you keep track of all the intermediate models, make them "vote" and take the majority.
↳ not memory efficient (you keep a lot of models)

↳ properties: i) Much better regularization performance than regular perceptron. they're almost as good as SVMs, and we can use the "kernel trick" - replace dot product with another kernel (we're not gonna get into that)

ii) Training is as fast as a regular perceptron

iii) Run-time is slower, (since we need n models)

**Average Perceptron**: (in practice, use this one)

One model, which is the average of all intermediate models

i) again, very simple, and we can use kernels.

ii) Exactly as fast to run and nearly as fast to train as regular perceptron

note: It's not mathematically the same to average 1000 weights than to let 1000 weights vote. Voting $\in \{1, -1\}$ (nonlinear procedure). If it was linear, (ex: linear regression), it would be the same

btw: "no one really uses perceptrons nowadays, but everyone covers it in ML classes"

"We had picked $\eta = \frac{1}{2}$. Is there a more clever way?"

**Passive-Aggressive Perceptron**:

↳ minimize then hinge loss at each observation:

$$\mathcal{L}(w_i \mid x_i, y_i) = \begin{cases} 0 & \text{if } y_i w_i^T x_i \geq 1 \\ 1 - y_i w_i^T x_i & \text{otherwise} \end{cases}$$

(loss = 0 if correct (i.e. if we're right) with margin $\geq 1$)

Pick $w_{i+1}$ to be as close as possible to $w_i$ while still setting the hinge loss to zero

⇒ if $x_i$ is correctly classified with a margin of at least 1, ⇒ no change $w_{i+1} \overset{?}{=} w_i$

else, $w_{i+1} = w_i + \eta\, y_i x_i$ where $\eta = \frac{\mathcal{L}(w_i \mid x_i, y_i)}{\|x_i\|^2}$

## Passive-Aggressive = MIRA

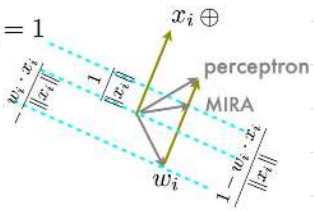
not super important

$$w_{i+1} = w_i + \frac{y_i - w_i \cdot x_i}{\|x_i\|^2} x_i$$

easy to show:

$$y_i(w_{i+1} \cdot x_i) = y_i \left(w_i + \frac{y_i - w_i \cdot x_i}{\|x_i\|^2} x_i\right) \cdot x_i = 1$$

new score $y_i (w_i \cdot x_i + y_i - w_i \cdot x_i) = y_i\, y_i$




Moves hyperplane so that new point is on the margin

### Margin-Infused Relaxed Algorithm (MIRA)

- *Multiclass*; each class has a prototype vector
  - Note that the prototype *w* is like a feature vector *x*
- Classify an instance by choosing the class whose prototype vector is *most similar* to the instance
  - *Has the greatest dot product with the instance*
- During training, make the 'smallest' change to the prototype vectors which guarantees correct classification by a specified margin
  - "passive aggressive"